The Project Chrono plant

How the multibody sedan behind `--dynamics chrono` works, why stepping it needs a solver, where its time goes, and how to make it faster without changing the car.

Summary. The Chrono plant is Chrono's Sedan: 20 rigid bodies, 42 joints, springs and dampers, a 6-shaft driveline and four Pacejka tires, behind a thin adapter that makes it take the same input and report the same state as the bicycle model. Each 1 ms step solves a 239 × 239 linear system for the bodies' accelerations and the joints' internal forces; on this machine that takes **1.22 ms**, 88 % of it in the solver, so the plant alone runs at 0.8 × real time (61 ms per 50 ms world step, against 0.03 ms for the bicycle plant). The default solver is converged. A direct solver is not a substitute: it cannot represent the open differential and locks it. **Warm-starting the default solver and giving it a tolerance of 10^{-2} makes the plant 2.2 × faster (0.55 ms per step) with the same trajectory to 2 cm over the 5 s test.**

1. Where the plant sits

Every plant in the project implements the same interface (`sim/plant/interface.hpp`): `reset(x0)`, `observe()`, `saturate(u)`, `step(u, duration)` and `set_friction(mu)`. The controllers never see which plant they drive. The input is `u = [steering rate, Fx]`, the steering rate of the road wheels [rad/s] and the total longitudinal force [N]; the state is

$$x = [\ X,\ Y,\ \psi,\ U_x,\ U_y,\ r,\ \delta\ ]$$

the position of the centre of mass, the heading, the forward and lateral velocity in the body frame, the yaw rate and the road-wheel angle. The plant is used in two places: offline by `build/simulate --plant chrono`, and in CARLA by `ChronoDynamics` in `carla_bridge/world.py`, where Chrono computes the motion and CARLA only draws the car (its physics is switched off) at the pose, roll and pitch Chrono reports.

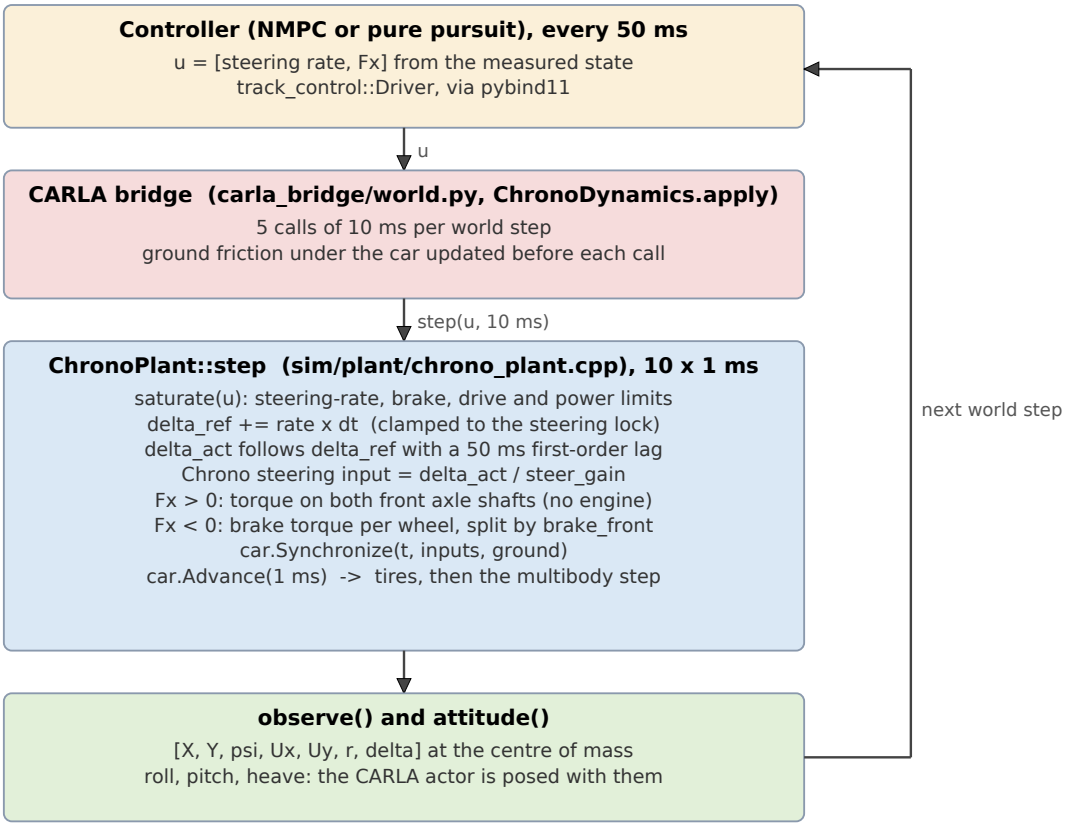


Figure 1. One 50 ms world step with `--dynamics chrono`. The car is stepped 50 times at 1 ms; the controller is called once.

2. The adapter: what ChronoPlant adds

Chrono's vehicle takes a driver's inputs (steering in [−1, 1], throttle, brake) and computes everything else. `ChronoPlant` (`sim/plant/chrono_plant.cpp`) turns the project's inputs into those, and Chrono's bodies back into the project's state.

Step	What it does	Why
saturate(u)	Clamps the steering rate to delta_rate_max and Fx between the brake limit and the smaller of F_drive_max and P_max / U_x.	The actuators' limits. The tires limit the force they can pass to the road themselves.
Steering	δ_{ref} += rate · dt, clamped to the lock; δ_{act} follows δ_{ref} with a first-order lag τ = 50 ms; Chrono's input is δ_{act} / steer_gain.	The controller commands a rate; Chrono's rack wants a normalised position.
Drive	Fx > 0 becomes a torque of ½·Fx·R on each front axle shaft (R the tire radius). Throttle stays 0.	Chrono's engine and gearbox are bypassed so the force is delivered as asked, within the parameters' limits.
Brakes	Fx < 0 becomes a brake torque on each wheel, split front/rear by brake_front, as a fraction of the 2000 N·m peak of Chrono's simple brake.	One input per axle instead of Chrono's single pedal.
Friction	The ground is flat; its friction μ is set before every 10 ms call from the surface under the car (asphalt or grass).	Chrono scales each tire's peak friction by μ / 0.8.
observe()	Position and velocity of the centre of mass, the heading from the chassis' x-axis, the velocity rotated into the body frame, Chrono's yaw rate, and δ_{act}.	δ is what a steering-column sensor would read; the wheels' true angle differs a little under load.
attitude()	Roll, pitch and the heave of the centre of mass.	Only for drawing: the CARLA actor is posed with them.

Table 1. From the project's input and state to Chrono's, in the order step() applies them.

Calibration when the plant is constructed

The plant measures its own car before the first lap, so nothing about the Sedan is hard-coded:

- **Ride height and centre of mass:** the car is built at rest and left to settle under gravity for 2 s (2000 steps) with the brakes on; then the chassis height and the centre of mass in the vehicle frame are recorded. The state is read at that point from then on.
- **Steering gain:** a steering input of 0.1 is held for 0.5 s and the mean angle of the front spindles' axes is measured; `steer_gain` = angle / 0.1, small enough for the linkage to be nearly linear.
- **Driveline ratios:** the car is rolled freely at 10 m/s for 1.5 s and the speed of each driveline shaft per unit of wheel spin is recorded. `reset()` uses those ratios to start the car at speed with the wheels and the driveline already turning, so the first steps do not jolt.

3. What Chrono simulates

The Sedan is assembled from Chrono::Vehicle templates. The listing below was read from the running model (`ChSystem::GetBodies`, `GetLinks`, `GetShafts`, `GetOtherPhysicsItems`).

Part	Bodies (mass)	Joints and force elements
Chassis	chassis (1515 kg)	free in 3D: rolls, pitches and heaves
Front suspension, double wishbone, ×2	upper arm (1.0 kg), lower arm (1.6 kg), upright (1.4 kg), spindle with the wheel (25.8 kg)	revolute joints at the chassis, spherical joints at the upright, spindle revolute, tie-rod as a distance constraint, spring and shock (TSDA)
Rear suspension, multi-link, ×2	trailing link (15.2 kg), lateral link (1.9 kg), upper arm (4.7 kg), upright (3.2 kg), spindle (25.8 kg)	universal joints at the chassis, spherical joints at the upright, upper-arm revolute, spindle revolute, tie-rod distance constraint, spring and shock
Steering, rack and pinion	rack link (1.9 kg)	prismatic joint, linear actuator driven by the steering input
Brakes	–	4 brake links on the spindle revolutes
Driveline (2WD, front)	– (1D shafts)	6 shafts (4 axle shafts, driveshaft, differential casing), 4 shaft–spindle couplings, a planetary gear as the differential, an angled gearbox, and a clutch that can lock the differential
Tires, Pac02 ×4	– (force elements)	Pacejka 2002 formula: force and moment at the contact patch from the slip and the load, applied to the spindle

Table 2. The Sedan's parts, with the masses Chrono reports.

Link type	Count	Constraint equations each
ChLinkLockRevolute (hinge)	10	5 (allows one rotation)
ChLinkLockSpherical (ball joint)	10	3 (allows all rotations)
ChLinkUniversal (Cardan joint)	4	4
ChLinkDistance (tie-rods)	4	1 (fixed distance)
ChLinkTSDA (springs and shocks)	8	0: a force, not a constraint
ChLinkLockPrismatic, ChLinkLockLinActuator (rack)	1 + 1	5, and 1 (rack position)
ChLinkLockBrake	4	a torque while sliding
Total	**42**	**113 bilateral constraint rows** with the driveline's

Table 3. The 42 links. Chrono reports 126 velocity coordinates (20 bodies × 6, plus 6 shafts) and 113 constraint rows, leaving 13 degrees of freedom.

The tires are not bodies. Pac02 is a semi-empirical model: from the longitudinal and lateral slip at each contact patch, the camber and the vertical load on that wheel, it returns the forces and the aligning moment, which Chrono applies to the spindle. Because every wheel has its own load, the load moves left to right in a corner and front to rear under braking, and the suspension geometry changes the camber and toe as the body rolls. None of that exists in the bicycle model, which lumps each axle into one wheel on a body that stays flat.

4. Why the dynamics need a solver

4.1 The bicycle plant: an explicit ODE

The bicycle model is written in **minimal coordinates**: one number per thing that can move. Every derivative is a formula of the current state and input, for example

$$dr/dt = (a \cdot F_{yf} \cdot \cos \delta - b \cdot F_{yr}) / I_{zz} \text{ , } dU_y/dt = (F_{yf} \cdot \cos \delta + F_{yr}) / m - r \cdot U_x$$

with the tire forces F_y from the Fiala model. Stepping it means evaluating $dx/dt = f(x, u)$ and integrating: no equation has to be solved, and 50 ms of driving takes 0.03 ms.

4.2 The multibody car: bodies held together by constraints

Chrono writes the car in **maximal coordinates**. Every body gets its own 6 degrees of freedom, as if it were floating free, and every joint removes some of them again with a constraint equation C(q) = 0. A ball joint between the upper arm and the upright, for instance, says that a point on the arm and a point on the upright are the same point: three equations. The equations of motion of the whole car are then

$$M \cdot dv/dt = f(q, v, t) + C_q^T \cdot \lambda \ , \ C(q) = 0$$

- **q, v:** the positions and orientations of all bodies and shafts, and their velocities (126).
- **M:** the mass and inertia of every body, block diagonal.
- **f:** every force that has a formula: gravity, springs, shocks, tire forces, the axle torque, aerodynamic drag, the gyroscopic terms.
- **C_q = ∂C/∂q:** the constraint Jacobian, how each joint equation changes as the bodies move.
- **λ:** one Lagrange multiplier per constraint row: the reaction forces and torques inside the joints.

The multipliers are why a solver is needed. There is no formula for the force in the ball joint of a wishbone: it is whatever force keeps the joint together while the tire pushes on the upright, the spring pushes on the arm and the chassis rolls. It depends on every other force and on every body's acceleration at the same instant, and they depend on it. The accelerations dv/dt and the forces λ can only be found together, by solving the two equations simultaneously. Mathematically the model is a differential-algebraic equation (DAE), not an ODE.

4.3 Why not minimal coordinates for the car too?

- **Closed loops.** A double wishbone is a closed kinematic loop: chassis, upper arm, upright, lower arm, back to the chassis; the tie-rod closes another one through the rack. Open chains (a robot arm) have obvious independent coordinates, the joint angles; closed loops do not, and the relation between them is itself a set of nonlinear equations to solve. Maximal coordinates with constraints describe any mechanism the same way, which is why general multibody engines use them.
- **Stiffness.** The suspension springs, the tires' vertical stiffness and the rigid joints act on millisecond time scales. An explicit integrator would need far smaller steps to stay stable, so Chrono integrates implicitly, and an implicit step means solving equations no matter which coordinates are used.

4.4 What is solved every millisecond

Chrono's default time stepper, `EULER_IMPLICIT_LINEARIZED`, is a semi-implicit Euler step linearised so that each step needs exactly one linear solve. With step h = 1 ms it finds the change of velocity Δv and the joint impulses hλ from the block system of Figure 2, then updates v and q. The −C/h term on the right is constraint stabilisation: if rounding has opened a joint slightly, it is pulled shut over the next step.

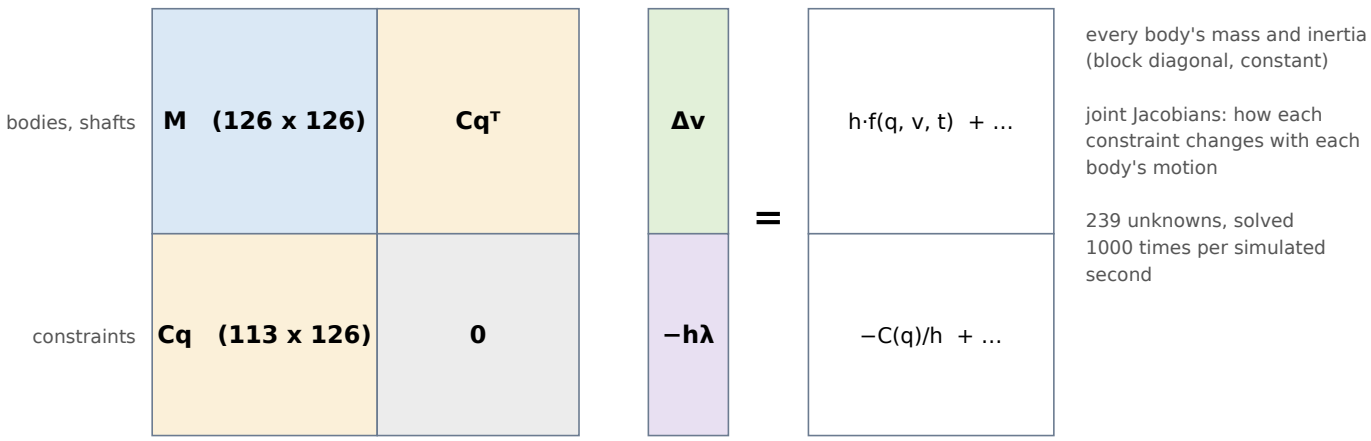


Figure 2. The linear system of one step, drawn to scale: 126 rows for the bodies and shafts, 113 for the constraints. It is solved 50 times per world step.

Two properties of this system decide which solver works:

- **It is badly conditioned.** The chassis weighs 1515 kg and the front upper arm 1.0 kg, a ratio of 1500 in the same matrix, and the joints are rigid. Iterative methods converge slowly on such systems.
- **It is not purely linear.** The differential clutch (`ChShaftsClutch`) is a boxed constraint: it holds only while the torque it carries stays within a limit, and slips beyond it. A boxed constraint turns the step into a variational inequality: solve the system, but with some multipliers kept inside bounds. The Sedan's clutch has limits ±100 N·m and modulation 0, so it transmits no torque at all: the differential is open, as the car is designed.

5. Two kinds of solvers

	Iterative VI solvers (Barzilai–Borwein, PSOR, APGD)	**Linear-system solvers (SparseLU, SparseQR, Pardiso, MINRES)**
How	Start from a guess and improve it; each iteration costs a few sparse matrix–vector products.	Solve the linear system as it stands: the direct ones factorise the matrix and solve it in one pass (the cost is the factorisation), MINRES iterates on it.
Boxed constraints	Yes: the bounds are projected at every iteration (these are VI solvers).	No. Chrono's ChDirectSolverLS: “cannot handle VI and complementarity problems”. A boxed constraint is treated as an ordinary equality: the clutch becomes rigid.
Accuracy	Depends on the iterations; slow on badly conditioned systems.	Exact for the equations it is given.
Chrono::Vehicle's default	Barzilai–Borwein, 150 iterations, tolerance 0 (always runs all 150), no warm start.	–

Table 4. The two families. Only the first can represent the open differential.

6. Measurements

A small C++ program built the Sedan exactly as `ChronoPlant` does (SMC contact method, no collision shapes, Pac02 tires, flat ground with μ = 0.9) and drove it for 5 s from 16 m/s with throttle 0.2 and steering 0.3·sin(t), at h = 1 ms, timing it with Chrono's own timers. Machine: Google Cloud VM, Intel Xeon at 2.20 GHz, 16 vCPUs, NVIDIA L4; Chrono 9.0.1 built in Release with `-march=native`; CARLA running alongside. Times vary by about 5 % between runs. The reference answer is Barzilai–Borwein at 1200 iterations, final position (37.04, 41.54) m.

6.1 Where the time goes

Measurement	**Result**
Chrono plant, 50 ms world step, from Python (5 × step(u, 10 ms))	61 ms: 0.8 × real time, 16 Hz
Bicycle plant, same	0.03 ms: about 1660 × real time
Chrono, one 1 ms step, default solver	1.22 ms; 88 % in the solver, 11 % elsewhere in the system step, about 1 % in the tires and vehicle code
Threads (OMP_NUM_THREADS = 1, 2, 4)	no change: one core, CPU time = wall time

Table 5. In a CARLA run with `--dynamics chrono` the plant step shown by the rate panel is these 61 ms plus about 9 ms for CARLA's world tick: about 14 Hz, 12 Hz observed.

6.2 Iterations of the default solver

Max. iterations	ms per step	× real time	Final position [m]	Distance from reference
10	0.20	4.88	diverges (NaN)	–
30	0.36	2.75	(41.40, 40.06)	4.60 m
50	0.51	1.96	(37.29, 41.49)	0.25 m
100	0.93	1.08	(37.06, 41.54)	2 cm
150 (default)	1.26	0.79	(37.03, 41.55)	1 cm
300	2.32	0.43	(37.03, 41.54)	1 cm
600	4.51	0.22	(37.04, 41.54)	< 1 cm
1200	8.81	0.11	(37.04, 41.54)	< 1 cm

Table 6. The default 150 iterations is converged: more iterations move the end point by less than a centimetre. Below 100 the answer drifts by metres, and at 10 the step blows up.

6.3 Direct solvers lock the differential

Solver	ms per step	Final position [m]	Front wheel spin, left / right [rad/s]
Barzilai–Borwein, 150 (default)	1.22	(37.03, 41.55)	−36.07 / −32.81 (open differential)
SparseLU	1.34	(46.50, 36.58)	–
SparseLU, sparsity pattern locked	0.98	(46.50, 36.58)	−35.31 / −35.31 (locked)
SparseQR	4.83	(46.50, 36.58)	–
MINRES	3.30	(46.50, 36.58)	–

Table 7. Every linear-system solver agrees on an end point 10.7 m away from the reference. They agree because they all solve the same, different, problem: with the clutch treated as rigid, both front wheels turn at exactly the same speed through the corner.

> Agreement between solvers is not evidence of correctness when they share a modelling limitation. Here the three “exact” solvers agree with each other and are all wrong about the car; the iterative solver, which looked like the approximate one, is the one solving the intended problem. Its convergence has to be checked against itself (Table 6), not against a direct solver.

6.4 Making the default solver stop early

With tolerance 0 the solver always runs to its iteration cap. Two settings let it stop when it is done: a tolerance on the residual, and **warm start**, which begins each solve from the previous step's multipliers. One millisecond later the joint forces have barely changed, so the previous answer is an excellent first guess.

Settings (Barzilai–Borwein, cap 150)	Mean iterations	ms per step	× real time	From reference
default: tolerance 0, cold start	150.0	1.22	0.82	1 cm
warm start only	150.0	1.23	0.81	1 cm
tolerance 10^{-3}	131.6	1.11	0.90	1 cm
warm, tolerance 10^{-3}	85.0	0.76	1.32	1 cm
warm, tolerance $3{\cdot}10^{-3}$	70.1	0.66	1.52	2 cm
warm, tolerance 10^{-2}	57.5	0.55	1.82	2 cm
warm, tolerance $3{\cdot}10^{-2}$	51.6	0.54	1.85	2 cm

Table 8. Warm start alone does nothing (the solver still runs all 150 iterations) and a tolerance alone does little; together they stop it after 50–85 iterations with the same answer.

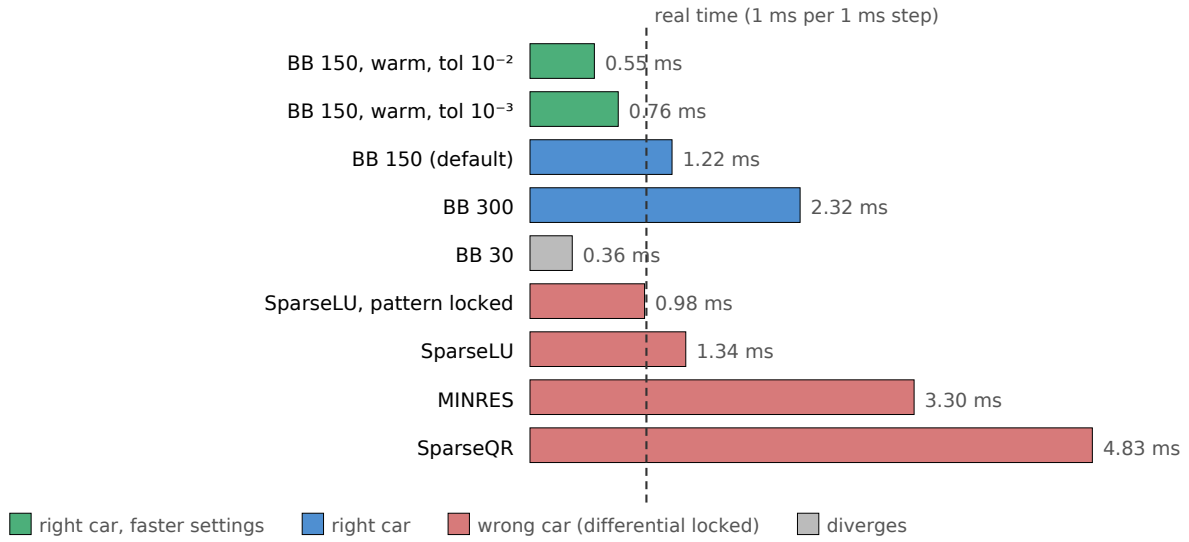


Figure 3. Cost of one 1 ms step. BB 30 is fast but drifts 4.6 m (grey: not usable). The time step itself should stay at 1 ms: with the default solver at 2 ms the end point moves 4.2 m.

7. Is the virtual machine the limit?

Only a little. The plant runs on one core and does not use threads, so what matters is the speed of a single core; a 2.2 GHz cloud Xeon is slower per core than a current desktop CPU, which might gain 1.5–2 ×, not the 8 × that separates the plant from 100 Hz. CARLA idling at about 3.4 of 16 vCPUs does not take that core away. The cost is in the model: 50 solves of a 239-unknown system per world step.

The NMPC matters more than the plant for the speed of a whole run: its solves take about 100 ms, so one world step with Chrono costs roughly 100 + 61 + 9 = 170 ms (6 Hz), and about 100 + 28 + 9 = 137 ms (7 Hz) with the faster solver settings. The runs are synchronous, so none of this changes the results, only the time a lap takes on the wall clock.

8. Recommendations

- **Warm-start the solver and give it a tolerance** in `ChronoPlant::Impl::build` after the car is initialised: `EnableWarmStart(true)`, `SetTolerance(1e-2)` on `car->GetSystem()->GetSolver()`, keeping the 150-iteration cap. Expected: 2.2 × faster, the plant faster than real time. Check it on full laps (`build/simulate --plant chrono`, both controllers, compared with the README's table) before relying on it; the test above is 5 s of one manoeuvre. Because the trajectory does not change, `config/vehicles/chrono_sedan.json` does not need re-identifying.
- **Do not switch to a direct solver** (SparseLU, Pardiso) while the driveline has the clutch: it locks the differential and changes how the car corners.
- **Do not cut the iteration cap** below about 100, or raise the step above 1 ms: both move the trajectory by metres within 5 s.
- **For overall speed, look at the NMPC first:** at about 100 ms per solve it is the larger share of a world step.

Appendix: the solver settings in code

```
// in ChronoPlant::Impl::build(), after car->InitializeInertiaProperties()
auto solver = std::dynamic_pointer_cast<chrono::ChIterativeSolver>(
    car->GetSystem()->GetSolver());
solver->EnableWarmStart(true);   // start from the previous step's joint forces
solver->SetTolerance(1e-2);    // stop when converged, at most 150 iterations
```

Not applied to the code yet: this document describes the measurements and the recommendation.